

DATOS DE LA ASIGNATURA	
Nombre(s) del Estudiante(s): Carlos Gualsaquí	Nivel: 3
Carrera: Desarrollo de Software	Docente: Ing. Santiago Chasiluisa
Asignatura: Lenguajes de programación 3	Periodo académico: ABR-SEP 2026

TAREA AUTONOMA

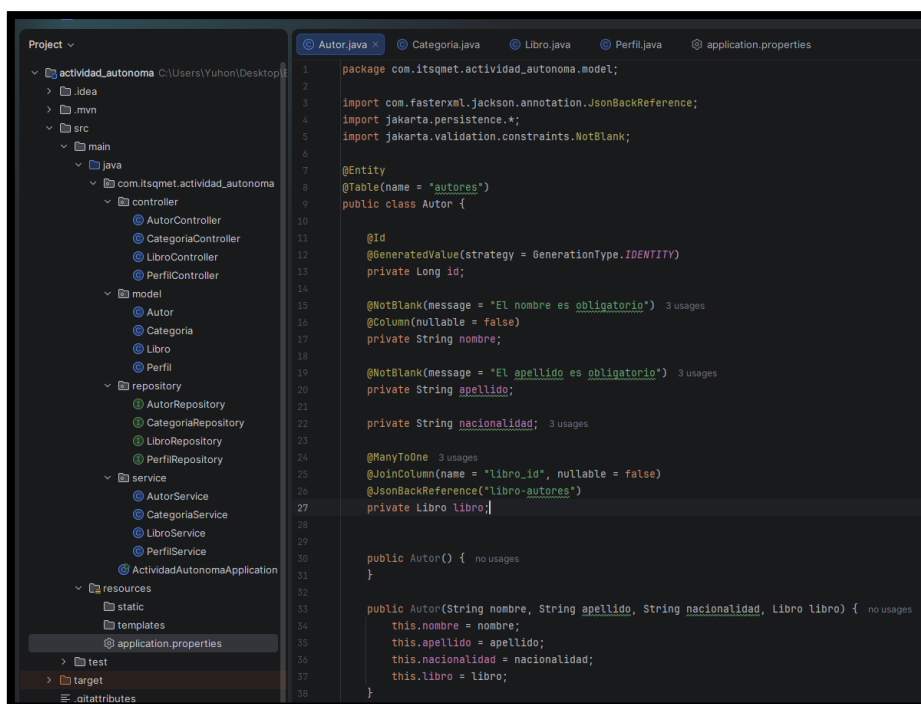
Tema: Implementación de una aplicación web a partir de un modelo de datos proporcionado.

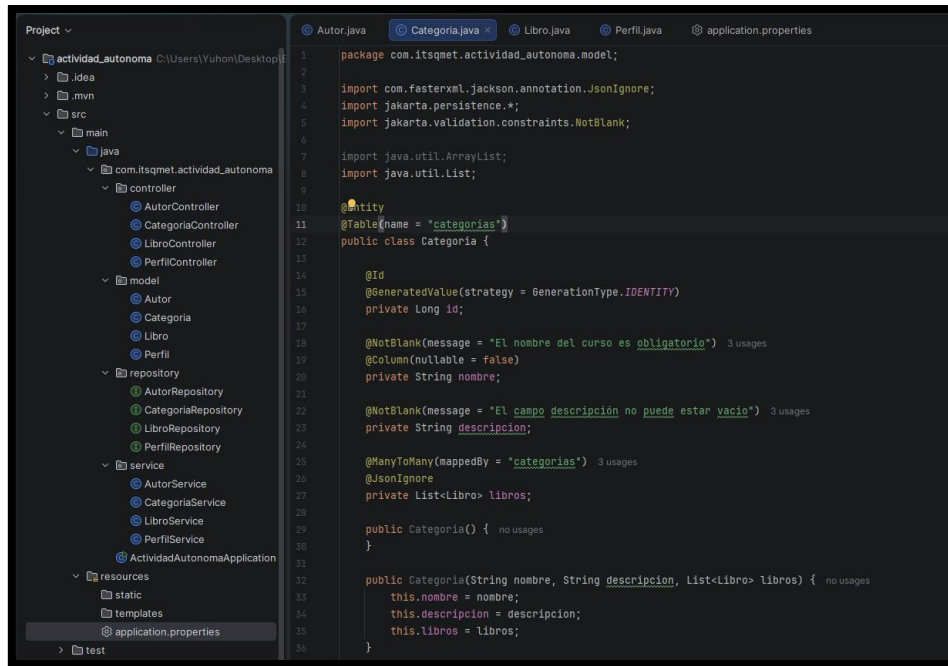
Descripción: Definir Implementar una aplicación web a partir de un modelo de datos proporcionado, desarrollando la estructura del sistema y la gestión de la información conforme a las entidades y relaciones definidas. La solución deberá permitir la interacción con los datos mediante operaciones funcionales y una interfaz coherente, asegurando un manejo eficiente de la información.

Instrucciones:

1. Implementa el modelo de datos proporcionado en Spring Boot, respetando las entidades, atributos y relaciones definidas.
2. Aplica correctamente los tres tipos de relaciones entre entidades usando las anotaciones JPA correspondientes.
3. Desarrolla las operaciones CRUD completas (Crear, Leer, Actualizar y Eliminar) para todas las entidades del sistema.
4. Crea una interfaz web sencilla que permita visualizar y gestionar los datos de cada entidad.
5. Verifica que las operaciones estén correctamente conectadas a la base de datos y que la información se actualice de manera adecuada en la interfaz.
6. Entrega un informe técnico en formato .pdf aplicando el formato preestablecido, que incluya el enlace al repositorio de GitHub.

• Back-end realizado en Intel IDEA

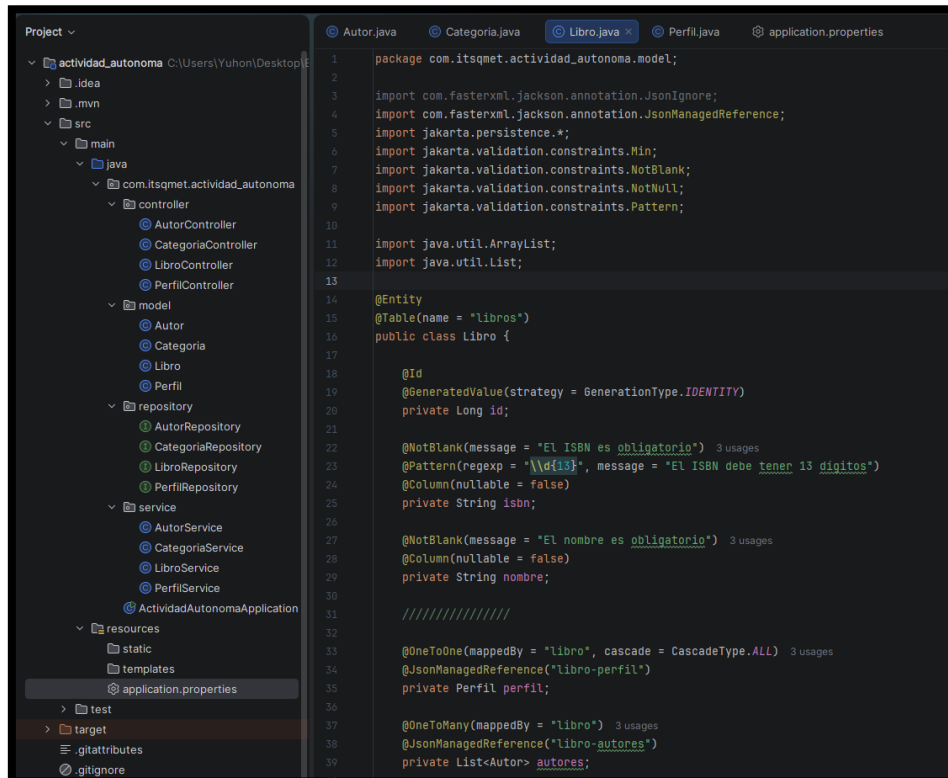




```

1 package com.itsqmet.actividad_autonoma.model;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import jakarta.persistence.*;
5 import jakarta.validation.constraints.NotBlank;
6
7 import java.util.ArrayList;
8 import java.util.List;
9
10 @Entity
11 @Table(name = "categorias")
12 public class Categoria {
13
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private Long id;
17
18     @NotBlank(message = "El nombre del curso es obligatorio") 3 usages
19     @Column(nullable = false)
20     private String nombre;
21
22     @NotBlank(message = "El campo descripción no puede estar vacío") 3 usages
23     private String descripcion;
24
25     @ManyToMany(mappedBy = "categorias") 3 usages
26     @JsonIgnore
27     private List<Libro> libros;
28
29     public Categoria() { no usages
30     }
31
32     public Categoria(String nombre, String descripcion, List<Libro> libros) { no usages
33         this.nombre = nombre;
34         this.descripcion = descripcion;
35         this.libros = libros;
36     }

```

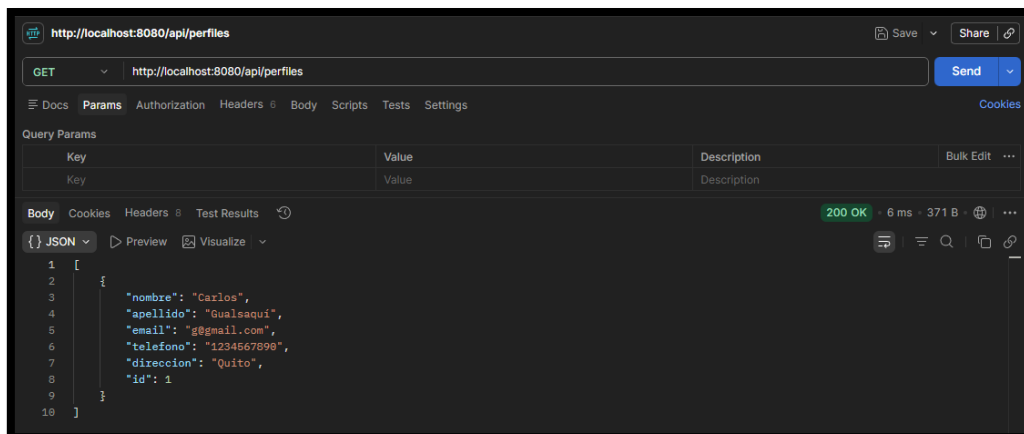
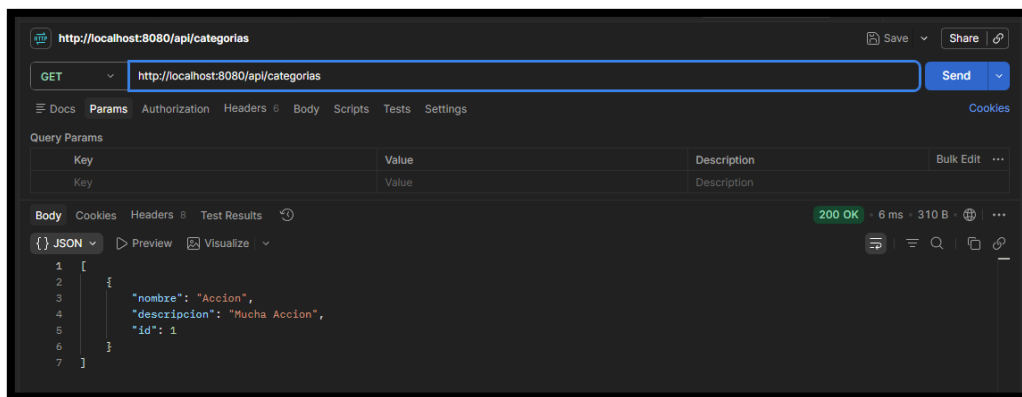
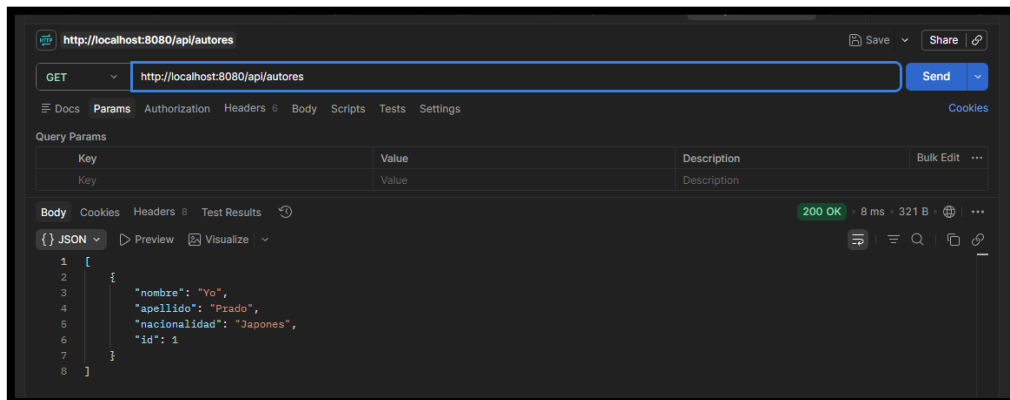


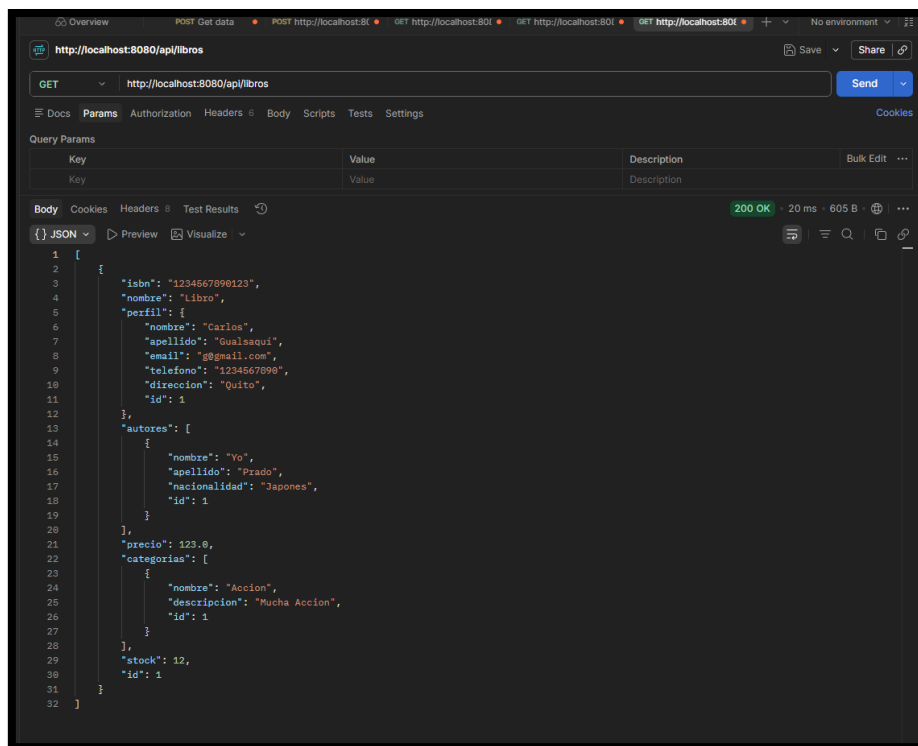
```

1 package com.itsqmet.actividad_autonoma.model;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import com.fasterxml.jackson.annotation.JsonManagedReference;
5 import jakarta.persistence.*;
6 import jakarta.validation.constraints.Min;
7 import jakarta.validation.constraints.NotBlank;
8 import jakarta.validation.constraints.NotNull;
9 import jakarta.validation.constraints.Pattern;
10
11 import java.util.ArrayList;
12 import java.util.List;
13
14 @Entity
15 @Table(name = "libros")
16 public class Libro {
17
18     @Id
19     @GeneratedValue(strategy = GenerationType.IDENTITY)
20     private Long id;
21
22     @NotBlank(message = "El ISBN es obligatorio") 3 usages
23     @Pattern(regexp = "\\d{13}", message = "El ISBN debe tener 13 dígitos") 3 usages
24     @Column(nullable = false)
25     private String isbn;
26
27     @NotBlank(message = "El nombre es obligatorio") 3 usages
28     @Column(nullable = false)
29     private String nombre;
30
31     //////////////////////////////////
32
33     @OneToOne(mappedBy = "libro", cascade = CascadeType.ALL) 3 usages
34     @JsonManagedReference("libro-perfil")
35     private Perfil perfil;
36
37     @OneToMany(mappedBy = "libro") 3 usages
38     @JsonManagedReference("libro-autores")
39     private List<Autor> autores;

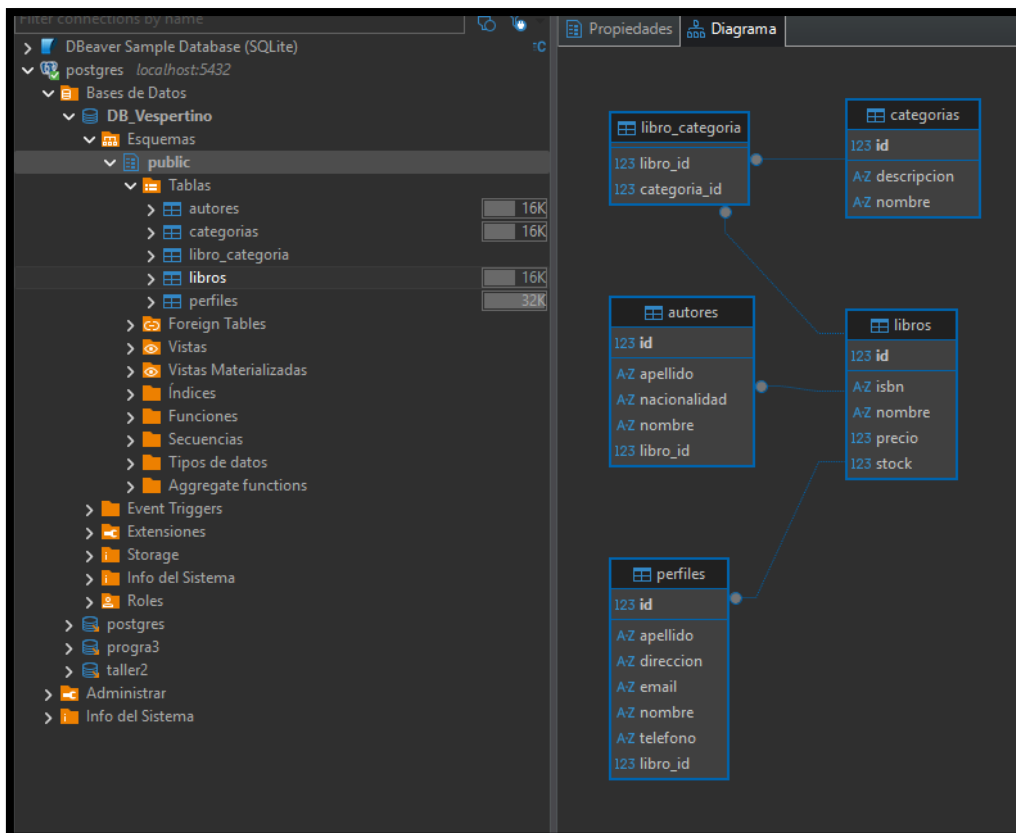
```

- Capturas del funcionamiento en Postman





- Base de datos realizada en DBeard



public categorías libros **perfiles** X

Propiedades Datos Diagrama

Show SQL Enter a SQL expression to filter results (use Ctrl+Space)

	123 id	AZ apellido	AZ direccion	AZ email	AZ nombre	AZ telefono	123 libro_id
Grilla	1	1	Gualsaquí	Quito	g@gmail.com	Carlos	1234567890

public categorías **libros** X

Propiedades Datos Diagrama

Show SQL Enter a SQL expression to filter results (use Ctrl+Space)

	123 id	AZ isbn	AZ nombre	123 precio	123 stock
Grilla	1	1	1234567890123	Libro	123

public categorías X libros

Propiedades Datos Diagrama

Show SQL Enter a SQL expression to filter results (use Ctrl+Space)

Tabla Propiedades

	123 id	AZ descripcion	AZ nombre
Grilla	1	1	Mucha Accion

- Front-end realizado en Visual Studio Code

Biblioteca Libros Autores Categorías **Perfiles** Nuevo perfil

Perfiles

ID	NOMBRE	APELLIDO	EMAIL	TELEFONO	DIRECCION	LIBRO	ACCIONES
1	Carlos	Gualsaquí	g@gmail.com	1234567890	Quito	—	Editar Eliminar

Nuevo perfil

X

NOMBRE (4-10 CARACTERES)

APELLIDO (4-15 CARACTERES)

EMAIL

TELEFONO (10 DIGITOS)

DIRECCION

LIBRO

Sin libro

Cancelar

Guardar

Biblioteca

Libros

Autores

Categorias

Perfiles

Categorias

Nueva categoria

ID	NOMBRE	DESCRIPCION	ACCIONES
1	Accion	Mucha Accion	

Editar

Eliminar

Nueva categoria

X

NOMBRE

DESCRIPCION

Cancelar

Guardar

Biblioteca

Libros

Autores

Categorias

Perfiles

Autores

Nuevo autor

ID	NOMBRE	APELLIDO	NACIONALIDAD	LIBRO	ACCIONES
1	Yo	Prado	Japones	—	

Editar

Eliminar

Nuevo autor X

NOMBRE

APELLIDO

NACIONALIDAD

LIBRO

Sin libro

Cancelar

Guardar

Biblioteca Libros Autores Categorías Perfiles

Libros Nuevo libro

ID	ISBN	NOMBRE	PRECIO	STOCK	CATEGORIAS	ACCIONES
1	1234567890123	Libro	\$123.00	12	Acción	Editar Eliminar

Nuevo libro X

ISBN (13 DIGITOS)

NOMBRE

PRECIO

STOCK

CATEGORIAS

Ninguna

Agregar categoria...

Cancelar

Guardar

Conclusiones

- El uso de anotaciones como @NotBlank, @Email, @Pattern y restricciones de tamaño asegura que los datos ingresados cumplan con reglas mínimas de calidad.
- Se implementó un CRUD completo para las entidades principales: Libro, Autor, Categoría y Perfil. Esto asegura que el sistema pueda administrar de manera eficiente la información relacionada con la biblioteca
- El frontend desarrollado con HTML, CSS y JavaScript ofrece una interfaz sencilla pero ordenada, que permite realizar operaciones CRUD de manera directa.
- La base establecida permite ampliar funcionalidades, como filtros de búsqueda, reportes, o integración con sistemas externos.

Enlace GitHub

https://github.com/Yuhon/Actividad_Autonoma_Progra3.git